

CS3000 Final Project

Sara Vasudev & Gavin Rossi

June 2026

1 Problem Domain

In a high-rise building, elevators are a shared, limited resource that must be distributed among many users competing for them at the same time. When several people on different floors all want elevator service, the building has to decide which elevator goes where. A common inefficiency is that users press every call button at once, causing multiple elevators to arrive at the same floor, while others sit idle, wasting the resource. We have designed and proved correct a greedy-dispatch algorithm to minimize rider wait time by allocating a single elevator to each button call so that the resource is used efficiently and wait times stay low. The core domain of this algorithm is resource allocation, where we allocate elevators (the resource) to the floors that need them.

2 Problem Specification

2.1 Input

The input of our algorithm is a single button call on each floor(i.e. a list of calls), plus the current state of all elevators in a high-rise building(i.e. a list of elevators). Each button call gives us four attributes of information. First, the floor where someone pressed the button(call.floor). Second,the direction they want to go – up or down(call.direction). Third, how long they’ve been waiting(call.wait). Fourth, if they’ve been assigned an elevator yet or not(floor.status). For each elevator, we track four things. First, the elevator’s current floor(elevator.floor). Second, the elevator’s direction – up, down, or idle (elevator.direction).

2.2 Output

The output of our algorithm is a set of 2-tuples {(elevator, call)} for each elevator dispatched by the algorithm. Each tuple shows a pairing of an elevator and the next call they are being dispatched to. While the idea of the algorithm is for each elevator to move towards the longest waiting and picking up the calls in between,the output only shows the very next stop which is not necessarily the longest-waiting.

2.3 Objective

We are optimizing the dispatch of multiple elevators in a building. The original problem we wanted to solve was buildings having a button for each elevator on a floor instead of one central button that dispatches the fastest elevator. However, to have an effective elevator network that only has one central button per floor, it is imperative that there is an algorithm that dispatches multiple elevators in the most efficient manner. We are optimizing for minimum wait-time.

2.4 Assumptions

Our assumptions for this algorithm are that a building has at least 2 floors, and at least 1 elevator. Additionally, we assume that the number of elevators in a building is both constant and within a small bound (2-4 is common, with the maximum being only 68 in one building) allowing iteration through them in $O(1)$ time. We are ignoring the potential impact of acceleration/deceleration between stops on overall time and the possibility for overloading of the elevator. Additionally, we're assuming one's wait-time begins when they press the call button and ends when the elevator picks them up. Lastly, we're assuming the amount of time an elevator takes to pass a floor is shorter than the time it takes to pick people on the floor.

3 Algorithm Design

3.1 Algorithmic Approach

We use a greedy algorithm supported by a min-heap and max-heap data structures. Our approach is greedy because for each button press, we make the locally optimal choice. We send whichever elevator can arrive the fastest, without reconsidering past decisions, to the floor with the button press. This gives each caller the shortest possible wait and keeps the algorithm fast, which we prioritize over being perfectly globally optimal.

The min-heap in our algorithm is keyed on each elevator's estimated arrival time, so the fastest elevator is always sitting at the root and can be pulled off in one step. This is our normal dispatch protocol. The max-heap is keyed on how long each outstanding call has been waiting, so the longest-waiting call is always at the root. Our max-heap is important, because while our greedy min-heap is predominantly what is running, if a call's wait time crosses a threshold, the max-heap surfaces it and forces it to be served next. This prevents a single caller from being skipped endlessly, and falls into our adversarial input and fairness constraint solution.

3.2 Overview in Plain Language

When someone presses the button on a floor, that call is timestamped so we can track how long it waits and added to a max-heap based on how long they've been waiting. Each subproblem is deciding the next best elevator, call pair. We use a total of three heaps: a max heap of calls with the longest waiting call at the root, a min-heap of elevators organized by their distance away from the given call (only contains elevators that are moving in the same direction as the call and has not passed the call), and a min-heap of call organized by their distance away from the given elevator (only contains calls that are moving in the same direction and the elevator has not passed it).

For each subproblem, we take the current longest-waiting call from the max-heap of calls' wait times, then take the closest elevator to it from the Min-Heap of elevators moving, and lastly go to the call nearest to the elevator (that's en route to the priority call). The 2-tuple of the dispatched elevator and the call it's dispatched to is added to the set. In this case the serviced call is the floor nearest the elevator. If the max-heap indicates the longest-waiting call has been waiting past a set threshold, that call is served first with no stops in between, to prevent starvation and waste of resources. In this case, the serviced call is the longest-waiting call.

3.3 Detailed pseudocode

ELEVATORDISPATCH(C, E, n, k, t)

```
1  // C = callers, E = elevators, n = number of callers, k = number of elevators, t = threshold
2  // c.floor = what floor the call is from, c.direction = what direction the call wants to go
3  // c.waiting = how long they've been waiting, c.status = if they've been serviced
4  // e.floor = what floor the elevator is on, e.direction = what direction the elevator is going
5  queue = BUILD-MAX-HEAP( $C.wait, n$ )
6  let S = a new, empty set
7  max = 1
8  while  $k > 0$ 
9      target = EXTRACT-MAX(queue)
10     if target == NIL
11         return S
12     while target.status != "waiting"
13         target = EXTRACT-MAX(queue)
14         if target == NIL
15             return S
16     best = CLOSEST-ELEVATOR(target, E, k)
17     if best == NIL
18         continue
19     best.direction = target.direction
20     if target.wait  $\geq t$ 
21          $S \cup \{(best, target)\}$ 
22         best.status = "dispatched"
23          $k = k - 1$ 
24     else
25         nearest = CLOSEST-FLOOR(best, C, n)
26          $S \cup \{(best, nearest)\}$ 
27          $k = k - 1$ 
28 return S
```

CLOSEST-ELEVATOR(c, E, k)

```

1  // c = target floor, E is a list of elevators, k is the number of elevators
2  heap = an empty min-heap
3  for i = 1 to k
4      el = E[i]
5      if el.direction != c.direction && el.direction != "idle"
6          continue
7      if c.direction == "up" && c.floor - el.floor < 0
8          continue
9      if c.direction == "down" && el.floor - c.floor < 0
10         continue
11     if c.direction == "up"
12         el.distance = c.floor - el.floor
13     else
14         el.distance = el.floor - c.floor
15     INSERT-MIN(heap, el.distance)
16 return EXTRACT-MIN(heap)

```

CLOSEST-FLOOR(el, C, k)

```

1  // el = dispatching elevator, C is a list of calls, k is the number of calls
2  heap = an empty min-heap
3  for i = 1 to k
4      c = C[i]
5      if el.direction != c.direction
6          continue
7      if c.direction == "up" && c.floor - el.floor < 0
8          continue
9      if c.direction == "down" && el.floor - c.floor < 0
10         continue
11     if el.direction == "up"
12         c.distance = c.floor - el.floor
13     else
14         c.distance = el.floor - c.floor
15     INSERT-MIN(heap, c.distance)
16 return EXTRACT-MIN(heap)

```

4 Proof of Correctness

ElevatorDispatch correctly assigns each available elevator to either the longest-waiting eligible caller (if their wait time meets or exceeds threshold t), or to the closest pending floor otherwise, and returns the complete set S of these assignments. Initialization: Prior to the first iteration, no elevators have been assigned, so S is empty and k equals its original value. The number of pairs in S is $[\text{original } k - \text{current } k] = 0$, which is trivially true. This condition holds since no assignments have been made. Maintenance: Assume the invariant holds at the start of iteration i , meaning S contains $[\text{original } k - \text{current } k]$ correctly assigned pairs. During the iteration, we extract the maximum-wait caller from the queue (line 9). We skip any callers that are no longer "waiting" (lines 12-15), preserving the invariant since S is unchanged and k is unchanged. Once a valid caller

is found, Closest-Elevator finds the best available elevator (line 16). If none exists, we continue without modifying S or decrementing k , which preserves the invariant. Otherwise, we add one pair to S and decrement k by 1 (lines 21-23 or 26-27), so [original k - current k] increases by exactly 1, maintaining the invariant. The greedy choice of highest-wait caller and closest elevator ensures the assignment satisfies the invariant's optimality condition. Termination: The loop terminates when either $k = 0$ or the queue is exhausted and EXTRACT-MAX returns NIL. k decrements by exactly 1 on every iteration that does not hit a continuation, and the queue is finite with at most n callers, so the loop must terminate. At termination, S contains exactly [original k - final k] pairs. If $k = 0$, all elevators have been dispatched. If the queue empties first, all serviceable callers have been handled. In either case, every pair in S reflects the locally optimal greedy decision made at the time of assignment, and S correctly represents the full dispatch schedule.

5 Run-Time Analysis

We chose to analyze the worst-case run-time for our algorithm. We chose worst case because elevators are a highly dynamic situation where it is likely for every edge case to be reached. Because of this, it is important to understand how efficient the algorithm is in the worst possible scenarios.

Elevator-Dispatch			
Steps	Operation	Times-Run	Inner Time
5	Create the queue heap	1	$\Theta(n)$
8-9	Loop as long as there are elevators	n	
12-14	Clear queue of serviced calls and get the highest waiting call	n	$\Theta(1)$
16	Find the elevator closest to the call	n	$\Theta(k \log k)$
17	Check that there is an available elevator for this call	n	
20	Check if the max wait-time threshold has been passed	k	
24-27	Find the nearest floor to the elevator and dispatch the elevator	k	$\Theta(n \log n)$

When we add up all these steps we get: $T(n) = \Theta(n) + \Theta(1) + \Theta(k \cdot n \log n) + \Theta(k \cdot n \log n)$

Worst Case Time-Complexity: $O(n \log n)$

6 Modeling and Fairness Analysis

6.1 What is your algorithm optimizing for? Who benefits from this optimization, and who might not?

Our algorithm optimizes for the speed of the person who just pressed the button. Specifically, our algorithm optimizes the time it takes for an elevator to reach the floor where the call was made. Every time a button is pressed, our algorithm looks at all the available elevators and sends the one that can get there the fastest. It also avoids sending more than one elevator to a single call, which cuts down on the wasted trips that happen when multiple elevators show up to the same floor. This benefits a) the person who pressed the button, because they get the quickest possible elevator b) the building as a whole, because redundant trips reduce overall elevator congestion, energy use, and traffic, and indirectly shortens everyone's wait time, and c) building managers and engineers benefit because our algorithm is simple and fast to run. However, our algorithm only cares about the newest caller's arrival time. This is because it has no memory of how long anyone has already been waiting. This means a person on a far-off floor can keep getting passed over when a closer call comes in. People already inside an elevator can lose out, because their elevator might get diverted to pick up a new nearby caller, which makes their own ride longer. However, this is a natural part of a greedy-algorithm trade off. Making the fastest choice for each individual does not guarantee an equally fair result for everyone in the building.

6.2 Construct an Adversarial Input. Provide a specific example where your algorithm produces an undesirable or unfair outcome.

A specific example of where our algorithm produces an adversarial input is when one caller ends up stuck waiting endlessly while everyone else gets served. During high traffic time with large amounts of clustering, wait-times for those furthest away greatly increase. Imagine a 20 story building where everyone on floors 2-19 are trying to take the elevator up (e.g. a conference on the top floor) and there exists elevator1 idle on floor 1 and elevator2 idle on floor 2. While the nearest elevator2 is dispatched towards the longest-waiting call, it is stopping at every call on the way. This means the call on floor 19 is waiting for elevator 2 to stop at every floor while elevator 1 remains idle on the bottom. Not only does this starve floor 19, but it also wastes the resources of the extra elevator.

6.3 Add a Fairness or Equity Constraint. Modify your problem to include a clearly-defined fairness requirement.

We added a fairness constraint in the form of a max wait-time. If the longest-waiting call has been waiting for an elevator longer than t time, the closest elevator is immediately dispatched without stopping in between. This solves the problem fully by eliminating the worst case scenario. Now if the above scenario ever happens, after t amount of time elevator2 would go straight to floor 19 and elevator1 would then start handling the rest that were in between. This fully stops starvation and resource waste.

6.4 Algorithmic Impact. Does your original algorithm still work, with the same run-time after adding this constraint? If not, what changed and why?

This fits perfectly within our algorithm and can even create a better time complexity. Checking the call's wait-time attribute against a given threshold is only $O(1)$ and if every call is over this max value, lines 24-27 are never run creating a best-case time complexity of $\Omega(n)$.